

《真神之死：终焉》卡牌 PVE 游戏作业报告

WORLD Alpha 0.9 / PVE 极限测试版 / 第三版整理稿

项目类型：Qt/C++ 单机剧情向卡牌 PVE 游戏

队伍：助教分配19队

小组成员：刘欣宇，高晟越，王艺戈

一、项目概述

本项目是一款以“真神消失、祝福与诅咒互相转化、主角追问世界起源”为叙事核心的单机卡牌 PVE 游戏。项目最终实现形态为 Qt/C++ 桌面程序，采用棋盘单位、技能卡、遗物、章节 Boss、剧情打字机 and 世界地图推进等机制，将早期卡牌战斗原型扩展为完整的九章十关式流程。

需求文档最初把目标定义为“实现完整 PVE 流程”，并明确不实现联网 PVP、本地 PVP、排名系统、商城系统、音乐系统和复杂动画特效系统。这使项目范围非常清晰：优先级为剧情体验高于战斗体验，战斗体验高于数值平衡，美术表现服务于剧情和可运行性。第三版不是追求商业级平衡的卡牌游戏，而是一个可通关、可演示、叙事闭环清楚的课程项目。

游戏名称在视觉与剧情文档中确定为《真神之死：终焉》。标题界面通过“祝福、诅咒、真神、邪神、以及一个问题”这一组关键词，将玩家提前带入主题：世界的规则曾由神维护，但真神已经消失；所谓赐福与诅咒并不是简单的善恶，而是推动角色选择、阵营冲突和最终问题的叙事装置。

项目当前版本以“卡牌游戏第三版”为 release 交付版本。它并不是一性写成的，而是经历了立项文稿、初级版、极限测试、极限测试 2、第二版整合以及第三版重构的迭代：前期解决“能不能跑”，中期补齐九章十关和剧情闭环，第三版再集中修整遗物、状态、标题界面、地图/剧情立绘映射以及回合顺序等细节。

项目要素	实现情况	说明
运行平台	Windows 桌面	使用 Qt Creator 打开 WorldPveExtreme.pro 或 CMake 工程运行
主要语言	C++ / Qt	界面、事件、动画、JSON 读取和战斗逻辑均在 Qt 框架内实现
核心模式	单机 PVE	无联网、商城、排行榜，所有资源本地加载
流程规模	序章 + 第一章至第八章	共 9 章，每章按 10 关或特殊剧情事件推进
当前定位	极限测试版 / 第三版	优先保证完整流程、可通关、可演示和剧情闭环

- # 小组成员
 - 刘欣宇（组长）、王艺戈、高晟越
- > 部分代码由 AI 协助完成
- # 前期准备
 - 开发方向讨论：刘欣宇、王艺戈、高晟越
 - 类与时序设计初稿：刘欣宇
- # 主体施工
 - ## 逻辑
 - 初期测试版：刘欣宇
 - 代码整理与重构：高晟越
 - 遗物系统设计：刘欣宇、高晟越
 - 技能系统设计：刘欣宇、高晟越
 - 状态系统设计：高晟越
 - ## UI
 - 剧情插图：刘欣宇
 - 文案：刘欣宇
 - 卡牌UI：刘欣宇、高晟越

- 战斗UI效果：高晟越
- # 后期调试
- 程序功能调试：刘欣宇、王艺戈、高晟越
- 平衡性调整：刘欣宇、高晟越

二、程序功能介绍

2.1 主流程与关卡推进

程序启动后先进入标题界面，背景图来自 assets/story/prologue/title.png，并通过淡入文本呈现作品主题。点击“开始游戏”后进入序章剧情，随后按章节顺序进入战斗、奖励、剧情、地图点亮和下一关。普通章节主要由 10 个关卡组成，第 5 关承担小 Boss 或剧情 Boss 功能，第 10 关为章节 Boss。第八章“世界”则被处理为特殊事件章节，以问答、遗物融合、最终问题和结局演出代替普通刷怪流程。

每一关的推进逻辑大致为：进入关卡时判断是否需要播放章节标题或地图覆盖层；读取 story.json 中对应 key 的剧情；布置敌我棋盘；玩家可选择技能卡并释放；点击回合按钮后执行战斗结算；胜利后发放金币、棋子、遗物或剧情奖励；如果存在战后剧情则继续播放；最后推进到下一关或下一章。失败时程序提供从本大章节检查点或从头挑战的恢复路径，避免演示时因一次失败直接中断完整流程。

2.2 棋盘、单位与主角成长

棋盘沿用了早期卡牌原型中的 5 格结构：前排 2 格、中排 2 格、后排 1 格。敌我双方各有一块 5 格棋盘，在 UI 中合计显示 10 张棋盘卡。主角固定存在于己方中排左侧，即 GameTypes.h 中的 kHeroSlot = 2，不能出售、不能移除。这个规则保证了剧情主角和战斗主体始终一致，也减少了死亡、补员和队伍替换时的异常情况。

主角属性随章节成长，从第一章的 30 生命/5 攻击逐步提升到第八章的 250 生命/70 攻击；第三版还加入永久攻击与生命修正，例如“染血符咒”可增加跨战斗攻击，“死亡圣契”则以生命上限为代价换取全体回满。这样主角成长不只来自固定章节表，也能被遗物选择影响。

2.3 技能卡与回合操作

技能卡由场上存活单位定期生成，最多保留 5 张，即 kMaxSkills = 5。玩家每回合最多选择 2 张释放，释放后触发治疗、护盾、单体伤害、群体伤害、召唤、状态叠加等效果。技能卡记录 name、source 和 faction，第三版新增 faction 字段，使“圣河水滴”等阵营相关遗物能够按天使阵营而不是按技能名匹配，减少硬编码判断带来的维护成本。

UI 上，技能槽不再只是简单按钮，而是 QFrame + QLabel 组成的卡牌结构。每张技能卡显示来源单位、技能名称、技能描述和回响标识；若来源棋子带有“回响”状态，卡牌底部会显示“回响×N”，提醒玩家该技能将多次释放。

2.4 遗物、奖励与融合

遗物是项目中最重要 的构筑系统。早期需求设定遗物槽最多 5 个，普通关胜利后随机展示 3 个遗物，玩家选择其中 1 个；剧情 Boss 则可能固定发放剧情遗物或剧情棋子。第三版将遗物槽扩展为 7 个，并用 RelicInstance 记录遗物名称、剩余使用次数和是否不占槽。遗物效果覆盖回合开始、战斗开始、受到伤害、造成伤害、战斗胜利、Boss 关前购买等多个时机。

剧情遗物构成了第八章“世界”机制的关键。需求文档中设定：当玩家同时拥有“跃动赤心、不死诅咒、圣洁六翼、邪神赐福”时，自动融合为“世界”。第三版中 tryFuseWorld() 负责检测这一条件，融合后代表主角收束前面章节的祝福/诅咒碎片，进入最终问题与结局演出。

系统	玩家看到的功能	代码中的支撑
棋盘单位	敌我十格卡牌、生命条、护盾、攻击、状态	UnitTemplate、UnitInstance、BoardCard
技能系统	最多 5 张技能，每回合最多释放 2 张	SkillCard、generateSkills()、castSkill()

系统	玩家看到的功能	代码中的支撑
遗物系统	最多 7 个遗物，胜利三选一，部分遗物有次数	RelicInstance、addRelic()、showRelicChoice()
状态系统	石化、脆弱、荆棘、回响、铸剑	StatusEffect、addStatus()、tickStatuses()
剧情系统	白屏打字机、角色名、立绘、点击推进	StoryManager、story.json
地图系统	章节标题、世界地图点亮、结局白幕	MapManager、assets/maps

三、需求与剧情设计

本项目不是单纯的数值对战，而是“剧情先行”的 PVE 卡牌流程。程序设计上要求剧情触发时暂停战斗、隐藏棋盘、显示纯白覆盖层、底部黑色文本框、文字逐字打印，并支持点击跳过当前句或进入下一句。第三版的 StoryManager 正是围绕这些要求实现，让剧情播放成为主流程的一部分，而不是附加说明文字。

世界观的核心是“真神曾经存在，但已经消失；邪神并非只是反派，而是新规则的象征”。序章中魔王??? 告诉玩家“世界已经开始改变”“魔族将出现在任何地方”“真神已经消失”，这几句话奠定了后续章节的冲突。玩家一路经过人族、吸血鬼、精灵、天使、魔族和终焉之地，实际上是在收集各个阵营对祝福、诅咒、永恒和死亡的不同解释。

章节设计采用“一个阵营、一段记忆、一个 Boss、一件遗物”的方式串联。第一章通过阿拉贡表现人族守护与执念；第二章通过吸血鬼伯爵和偷窃者米格表现复制、窃取与记忆；第三章通过艾琳和跃动赤心表现生命冲动；第四章通过阿格尼表现精灵圣地与不死秘密；第五章通过米凯尔和加百列表现天使审判；第六章、第七章把魔境和终焉推向邪神赐福；第八章则把前面获得的碎片合成为“世界”，让玩家回答“什么让我们不再永恒”。

剧情文本没有只停留在文档层，而是被结构化为 data/story.json。第三版 story.json 约包含 96 个场景 key、457 行文本，其中大部分行绑定了立绘路径。这种做法使文案、立绘和程序触发点解耦：程序只需要按 key 播放场景，而具体台词、说话人和图片可以由脚本或 JSON 独立维护。

四、项目各模块与类设计细节

4.1 总体架构

第三版的架构已经从早期“主窗口里写全部逻辑”的原型，演进为几个边界较清晰的模块：GameTypes.h 负责常量、数据结构和章节静态配置；BattleEngine 负责战斗、技能、遗物、状态、Boss 机制、奖励和胜负结算；StoryManager 负责剧情加载和打字机覆盖层；MapManager 负责章节标题和世界地图覆盖层；MainWindow 负责 UI 构建、跨系统编排、刷新和用户事件。

这种拆分的意义在于降低耦合。战斗引擎不直接操作所有 UI 控件，而是通过 setLogCallback、setRefreshCallback、setRestartGameCallback 等回调与 MainWindow 通信。MainWindow 保留 chapterIndex、levelIndex、标题界面、按钮和卡牌控件，但不再承担全部战斗结算。StoryManager 与 MapManager 都作为 QObject 模块独立管理 overlay、定时器和 eventFilter，使剧情点击、地图点击和主窗口按钮逻辑不混杂在一起。

文件/类	主要职责	规模（约）
GameTypes.h	游戏常量、单位/遗物/状态/技能/章节结构、九章配置	149 行
BattleEngine.h/.cpp	战斗引擎、技能释放、遗物触发、Boss 机制、奖励结算	约 2619 行

文件/类	主要职责	规模（约）
MainWindow.h/.cpp	窗口布局、标题界面、UI 刷新、按钮事件、动画数字	约 1389 行
StoryManager.h/.cpp	story.json 加载、打字机、剧情 overlay、立绘缩放	约 338 行
MapManager.h/.cpp	章节标题、地图点亮、白幕淡出、overlay 点击	约 466 行
tools/*.py	地图素材生成、剧情 JSON 生成辅助工具	辅助脚本

4.2 GameTypes：统一的数据模型

GameTypes.h 是项目的数据模型层。它定义了 kHeroSlot、kMaxSkills、kMaxRelics 等常量，并定义 UnitTemplate、UnitInstance、SkillCard、RelicInstance、StatusEffect、ChapterDef 等结构。UnitTemplate 表示一种单位的静态属性，UnitInstance 表示战斗中的动态状态；前者包括名称、阵营、排位、技能、生命、攻击，后者再附加当前生命、护盾、存活回合数、状态列表、是否主角、是否 Boss、是否复活过等字段。

RelicInstance 和 StatusEffect 是第三版的重要扩展。RelicInstance 让遗物不再只是 QStringList 中的名字，而能表达永久遗物、有限次数遗物和一次性不占槽遗物；StatusEffect 用 name、layers、decays 三元组统一表达增益和减益，例如石化、脆弱、荆棘、回响、铸剑。这种简化结构比为每个状态写一个派生类更适合课程项目，也便于 UI 统一显示层数。

4.3 BattleEngine：战斗核心

BattleEngine 是项目体量最大的模块，承担全部战斗相关逻辑。它维护 enemyUnits_、playerUnits_、roster_、skillSlots_、relics_、logLines_、gold_、battleRound_、pendingCombatEvents_ 等状态，提供从单位创建、棋盘查询、伤害治疗，到技能释放、遗物触发、Boss 机制、胜负结算的一整套接口。

战斗结算中最基础的是 dealDamage()、heal()、addShield() 和 allAttack()。dealDamage() 会先扣护盾再扣生命，并根据脆弱、铸剑、荆棘等状态修正伤害或触发反伤；allAttack() 则遍历攻击方存活单位，选择目标并执行普通攻击。runRound() 将这些基础能力串联为完整回合：生成或释放技能、处理双方攻击、清理死亡、推进回合数、触发遗物和状态衰减，再判断胜利或失败。

Boss 机制集中在 bossMechanics()、showBossSkillIntro() 和 setupBattle()/setupWorldEvent() 中处理。不同 Boss 使用名称判断触发特性，例如偷窃者米格复制玩家单位并释放偷来的技能，阿格尼、米凯尔、伊维尔、莱索恩等拥有对应章节的特殊行为。虽然这些 Boss 机制仍带有“按字符串分支”的原型特征，但优点是直观、易维护，能够稳定支撑完整演示。

4.4 StoryManager：剧情打字机

StoryManager 封装了需求书中对白屏打字机的要求。loadStory() 从 data/story.json 读取全部场景，showStory() 和 showStoryKey() 按 key 播放，内部以 pendingStoryKeys、activeStory、storyLineIndex、storyCharIndex 和 storyLineComplete 维护状态机。tickStory() 每 25ms 增加一个字符，eventFilter() 捕捉 overlay 点击：如果当前行未播完，则立即补全；如果已经播完，则进入下一句或下一场景。

每条剧情行包含 speaker、text 和 image。applyStoryImage() 根据相对路径加载立绘，refreshStoryImage() 根据窗口尺寸保持图像比例。这样程序既能实现纯文字白屏，也能展示章节立绘和 Boss 图像。剧情播完后通过回调回到关卡流程，避免剧情系统自己直接修改 battleEngine 或 MainWindow 的关卡状态。

4.5 MapManager 与 MainWindow: 呈现层

MapManager 负责章节标题和世界地图两个 overlay。章节标题采用中文标题与英文标题淡入，地图 overlay 使用世界地图图片和 10

个坐标点，支持点亮动画和结局白幕淡出。mapPointForChapter()、mapPointRatio() 和 chapterEnglishName() 将章节索引映射到地图坐标和英文标题，使章节推进具有视觉反馈。

MainWindow 则是玩家直接接触的界面层。它构建标题界面、顶部状态、敌我棋盘、底部技能栏、右侧遗物栏和日志区域。第三版的 UI 比早期按钮文本更具体：主角状态使用生命/护盾进度条，棋盘卡显示排位、攻击、名称、生命条、护盾和状态；遗物卡显示名称、描述和剩余次数；技能卡显示来源、名称、描述和回响。animateCombatEvents() 还根据 BattleEngine 的事件队列播放浮动伤害、治疗和护盾数字。

五、核心流程与关键算法

5.1 关卡进入流程

进入关卡时，MainWindow 根据 chapterIndex 和 levelIndex 判断是否需要播放章节标题、地图点亮或剧情场景。普通章节会调用 BattleEngine::setupBattle() 布置敌人，再调用 preparePlayerForBattle() 继承己方单位状态并保持主角固定；世界章节则走 setupWorldEvent()，其中包含问答、特殊 Boss、遗物融合和最终问题。

setupBattle() 的设计重点是吧章节配置与战斗实例分开。章节配置来自 GameTypes.h 中的 chapters()，其中每一章定义普通敌人池、第 4 关精英、第 5 关 Boss、第 9 关精英、第 10 关 Boss、剧情遗物和剧情队友。setupBattle() 根据关卡号选择敌人模板并放入敌方棋盘，再触发战斗开始类遗物效果。

5.2 回合结算流程

回合结算以按钮驱动。玩家可以先选择至多两张技能，点击释放后技能立即结算；随后点击回合按钮，runRound() 处理普通攻击、Boss 行为、死亡清理、补员、遗物效果、状态衰减和胜负判断。第三版对回合顺序做过修正：applyRelicsStart() 不再在 runRound() 一开始触发，而是移动到战斗结算后、回合数推进之后，保证“每回合开始时”的效果不会在敌人尚未部署或本回合尚未结束时错误触发。

状态衰减统一由 tickStatuses() 完成，带 decays 的状态每回合减少一层，归零后移除。石化和脆弱等负面状态在敌方身上通常会衰减，荆棘、回响、铸剑等正面状态可根据设计选择不衰减。统一状态列表减少了跨函数传参，也方便 MainWindow 将正面状态显示为绿色、负面状态显示为红色。

5.3 遗物触发与次数管理

第三版遗物系统的关键改动是“名字列表”升级为“实例列表”。hasRelic() 替代原先的 relics_.contains()，relicUses() 查询剩余次数，consumeRelic() 负责次数减少并返回是否归零，cleanupZeroUseRelics() 负责统一清理。这样“断爪”“石像鬼雕像”等有限次数遗物、“许愿骨”“死亡圣契”等一次性遗物，以及“锈蚀胸甲”“铁玫瑰”等永久遗物可以共存于同一个列表中。

遗物槽从 5 个扩展到 7 个，既提高了构筑空间，也降低了剧情遗物挤占普通遗物的问题。剧情遗物仍可通过 isStory 标记影响替换对话框提示，但不再使用固定槽位的特殊逻辑。普通遗物奖励、许愿骨的连续三选一、黄金祭坛的金币购买都复用了 showRelicChoice()，减少重复 UI 流程。

5.4 剧情与结局触发

剧情触发点遵循统一命名：例如 c1_l1 表示第一章第一关前剧情，c1_l1_after 表示第一章第一关后剧情，boss_阿拉贡 表示 Boss 登场提示，ally_阿拉贡 表示伙伴加入剧情。程序在关卡前后检查 StoryManager::hasScene()，存在则播放，不存在则直接继续。这种 key 约定让剧情脚本可以不断补充，不必每次都改动 C++ 主流程。

最终流程是项目叙事闭环的重点。第八章先通过问答让玩家选择 Boss，再引导战斗和剧情遗物收束；当四件关键遗物融合为“世界”后，玩家面对“什么让我们不再永恒？”这一最终问题。程序不判断答案对错，而是直接进入结局：白色背景、逐句显示、返回主菜单。这个设计避免把主题简化成标准答案，而是把玩家输入当作进入结局的仪式。

六、版本演进与实现取舍

项目迭代路线比较清晰：立项文稿先确定 PVE、剧情优先和不做联网系统的范围；初级版验证基础棋盘、单位、技能和 Qt 窗口；极限测试版把“能完整通关”放在第一位，补齐序章到第八章、Boss、技能槽、遗物槽和结局；极限测试 2 与第二版整合阶段继续补充标题、地图、立绘和剧情流程；第三版则作为 release 版本，集中完善遗物、状态、标题界面、战斗展示、回合顺序和章节衔接等内容。

这个过程体现了一个比较现实的取舍：先保证主流程闭环，再逐步提高机制丰富度和显示质量。卡牌游戏很容易陷入数值平衡和效果扩展，但如果没有完整章节和结局，演示体验会很碎。当前第三版已经可以完整运行和通关，并且把后续可拓展的方向集中到了数值平衡、Boss 配置化和系统拆分上。

七、测试、运行与发布状态

项目推荐使用 Qt Creator 打开 WorldPveExtreme.pro，Kit 为 Desktop Qt 6.11.1 MinGW 64-bit。当前 release 以第三版为准，程序已经可以正常运行，具备本地演示、完整通关和打包提交的基础。

从测试角度看，项目至少覆盖了以下路径：启动标题界面、序章剧情、普通战斗、Boss 战、胜利奖励、剧情遗物、伙伴加入、地图点亮、第八章特殊事件、最终问题、结局返回主菜单。第三版开发过程中也做过针对性验证，例如检查战斗开始效果触发时机，保证断爪和石像鬼雕像能正确给已部署敌人添加状态；检查补员窗口逻辑，避免空候选窗口阻塞；检查伤害日志，使日志与动画数字一致。

测试点	期望结果	当前状态
标题界面	背景按比例铺满，关键词与开始按钮淡入	已实现
剧情打字机	逐字显示，点击可补全或推进	已实现
普通战斗	敌我攻击、技能、护盾、死亡清理正常	已实现
状态效果	石化/脆弱/荆棘/回响/铸剑按层数生效	第三版新增并验证关键路径
遗物系统	永久/有限/一次性遗物均可管理	第三版重构
世界章节	问答、Boss、融合、最终问题、结局串联	已实现
发布包	release 程序可运行，可用于演示和提交	已完成

八、项目总结与反思

8.1 完成度总结

从课程作业角度看，本项目的主要价值在于完成了从需求文档到可运行游戏的闭环。它不仅有基础战斗，还包含标题界面、剧情系统、章节系统、地图系统、Boss 机制、遗物构筑、状态效果、奖励选择、最终问题和结局。尤其是剧情文档与 story.json 的结合，使游戏的文本内容可以独立维护，不需要把所有台词写死在 C++ 代码中。

程序结构也有明显迭代。第一阶段更像单窗口实验，后续逐步拆分出 BattleEngine、StoryManager 和 MapManager。虽然 BattleEngine 体量较大，Boss 机制和技能机制中字符串分支较多，但相较早期把所有逻辑塞进 MainWindow 的方式，第三版已经具备了可解释、可维护、可继续扩展的架构。

8.2 主要收获

第一，项目证明了“先完成主流程，再做细节”的重要性。卡牌游戏很容易陷入数值平衡和效果扩展，但如果没有完整章节和结局，演示体验会很碎。极限测试版先把九章十关、剧情和结局跑通，第三版再修遗物、状态和 UI，这个顺序使项目始终有可展示成果。

第二，数据结构会直接影响后续扩展。遗物从 QStringList 改成 RelicInstance 后，次数、槽位、一次性和剧情标记都能自然表达；状态统一为 StatusEffect 后，石化、脆弱、荆棘、回响、铸剑可以共享 addStatus、removeStatus、statusLayers 和 tickStatuses。这些改动让后续新增遗物和状态时不用每次推翻旧代码。

第三，剧情系统的数据化非常关键。story.json 的 96 个场景和 457 行文本说明本项目的剧情量已经不小。如果全部写在 C++ 里，维护成本会很高；现在通过 key 触发和 JSON 配置，剧情、立绘和程序流程可以分工维护。即使本报告按要求忽略成员分工，代码层面的职责分离仍然是项目成功的重要原因。

8.3 不足与改进方向

项目后续可以继续优化架构。首先，BattleEngine 体量较大，技能、遗物、Boss、奖励和战斗流程都集中在一个类中，后续可以继续拆分为 SkillSystem、RelicSystem、BossSystem 和 RewardSystem。其次，Boss 机制大量依赖中文名称字符串判断，短期方便，长期维护时更适合使用枚举、ID 或配置表替代。

其次，平衡性可以继续打磨。当前技能和遗物效果以简化数值为主，优先服务完整流程。第三版加入大量新遗物和状态后，强度组合更多，例如回响、铸剑、世界复制技能等机制可能导致后期爆发过高；死亡圣契、染血符咒等永久修正也可以通过更多通关测试来校准。

最后，美术和交互仍可提升。当前已经有标题图、章节立绘、地图和卡牌 UI，但战斗动画、音效、教程提示、设置界面、存档读档、键盘操作提示等仍比较简化。若继续开发，建议优先加入存档/读档与难度选择，因为这两项会显著提升长流程 PVE 的可玩性和演示稳定性。

8.4 结语

《真神之死：终焉》从一个卡牌原型发展为完整 PVE 剧情游戏，最核心的成果不是某一个单独功能，而是“系统之间能够互相接住”：剧情能触发战斗，战斗能产出遗物，遗物能推动世界章节，世界章节能回到主题问题，最终问题再进入结局。这个闭环让项目具有完整作品感。

如果用一句话概括本项目：它是一款以 Qt/C++ 实现的剧情向卡牌 PVE 作品，在有限时间内优先完成了主流程、叙事表达和核心机制，并通过第三版重构把遗物、状态和 UI 稳定性向可发布版本推进。后续只要继续拆分战斗引擎、配置化 Boss 与技能、补充测试和调平，就可以在当前基础上成长为更成熟的独立小游戏。